

SOFT COMPUTING CAPSTONE

SC04

Adaptive Traffic Signal Optimization System

Beginner Project Guide

All steps at a glance, with a fully spoon-fed Step 1.

Team size: 3-4 students

Step 1 target: 3-5 working days

M1: one week before mid-semester examination

M2: one week before end-semester examination

What are you building?

Build a small traffic-junction simulator. The app should choose which direction gets green and for how long, using queue and waiting-time information.

Who will use it?

A traffic operator studying one simple two-phase junction.

One-sentence problem

Given queues, density, waiting time, and emergency direction, choose the next phase and green duration.

What M1 must show

A fuzzy controller for one two-phase junction, compared with a fixed-time signal.

What M2 must add

Use GA to tune the controller, test emergencies and heavy queues, and deploy the simulator.

Important

M1 must be a working Product V1: input -> processing -> output. Step 1 is the preparation needed before the main fuzzy, ANN, or GA algorithm. M2 must add a real technical improvement and a deployed Product V2.

Your eight-step project path

Step	Stage	What happens
1	M1	Define one simple two-phase junction.
2	M1	Generate and validate traffic states.
3	M1	Build the fixed-time signal and queue updates.
4	M1	Build fuzzy phase priority and green duration.
5	M1	Create the working junction Product V1.
6	M2	Use GA to tune controller parameters.
7	M2	Compare controllers in difficult traffic cases.
8	M2	Deploy Product V2 and finish all evidence.

Where this guide spends most of its time

The remaining pages explain Step 1 in very small, doable actions. Finish every checkbox before starting the main algorithm.

Do not begin fuzzy rules, neural-network training, or GA optimization just because one member is excited to code. First agree on the problem, fields, units, sample data, baseline, tests, and evidence.

What Step 1 must produce

- ☐ A GitHub repository that every member can open.
- ☐ A README.md that explains the project and how the team will work.
- ☐ A docs/STEP-1.md project contract.
- ☐ A data/README.md with field meanings, units, sources, and assumptions.
- ☐ The correct starter data files.
- ☐ Twenty clean starter rows or one small, complete fixture.
- ☐ Five named test cases, including an invalid or impossible case.
- ☐ A simple baseline written as numbered rules or pseudocode.
- ☐ A validation program that prints PASS for valid data and rejects bad data.
- ☐ A simple Product V1 screen sketch.
- ☐ At least one meaningful Git commit from every member.
- ☐ A three-minute faculty demonstration.

Suggested five-day plan

1	Day 1: create repository, add members, assign roles, and write the one-sentence problem.
2	Day 2: define every input/output field, unit, range, category, and missing-value rule.
3	Day 3: create the 20-row sample or small fixture; write the five test cases.
4	Day 4: write baseline rules, validation code, screen sketch, and make member commits.
5	Day 5: clone/run from the beginning, correct errors, and rehearse the faculty demo.

Create the working space

1. One member creates a new repository. Suggested name:

SC04-Adaptive-Traffic-Signal-Optimization-System-Team-Name

2. Add all members as collaborators. Every member accepts the invitation.

3. Add these folders and files:

```
README.md
requirements.txt
docs/STEP-1.md
data/README.md
data/sample_input.csv # or the project-specific starter files
src/validate_data.py
notebooks/
app/
results/
report/
```

4. Assign first responsibilities

Responsibility	Exact Step 1 job
Repository	Create folders, add members, and test clone/run steps.
Data	Write fields/units, make the sample, and record sources.
Baseline	Write simple rules and expected results for five cases.
Testing/UI	Write validation, draw the screen, and collect evidence.

For three students, combine Repository with Testing/UI. These are first responsibilities, not permanent silos. Everyone must understand the complete Step 1.

5. If using Git commands

```
git clone REPOSITORY-URL
cd REPOSITORY-NAME
git checkout -b your-name-step1
# create or edit your assigned files
git add .
git commit -m "Describe the real work you completed"
git push -u origin your-name-step1
```

If Git commands feel difficult, use GitHub Desktop. The required idea is the same: pull first, work on a small task, write a clear commit message, push, and let the team review.

Write docs/STEP-1.md

Copy these headings into **docs/STEP-1.md**. Then replace every instruction with your team's answer.

Step 1 Project Contract

Team and responsibilities

- Member name - first responsibility

One-sentence problem

Given queues, density, waiting time, and emergency direction, choose the next phase and green duration.

User of the product

A traffic operator studying one simple two-phase junction.

Inputs and units

Write the field table from the next page.

Outputs and units

next_phase (NS/EW) and green_duration_sec, with safe minimum and maximum values

Baseline method

Write the numbered baseline rules from this guide.

Soft Computing method for M1

A fuzzy controller for one two-phase junction, compared with a fixed-time signal.

Advanced method for M2

Use GA to tune the controller, test emergencies and heavy queues, and deploy the simulator.

Dataset/scenario sources

Add real URLs, licences, and assumptions.

Five mandatory test cases

Copy and complete the five cases in this guide.

Product V1 screen sketch

Insert a photo or exported image of the sketch.

Risks and assumptions

List what may be wrong, missing, simulated, or temporary.

Step 1 completion evidence

Add links to files, commits, and validation output.

Before saving

[] Replace generic words such as 'user' or 'data' with project-specific words.

[] Write units beside every number.

[] Mark temporary thresholds and assumptions as temporary.

[] Do not claim that simulated data is real data.

[] Do not paste a source URL without saying what information came from it.

Create the data dictionary

Field	Simple meaning	Type/unit	Starter rule
state_id	Unique traffic state	text	TRF-001, TRF-002, ...
queue_ns	North-south queue	vehicles	0 or more
queue_ew	East-west queue	vehicles	0 or more
density_ns_pct	North-south density	percentage	0 to 100
density_ew_pct	East-west density	percentage	0 to 100
max_wait_ns_sec	Longest NS wait	seconds	0 to 300
max_wait_ew_sec	Longest EW wait	seconds	0 to 300
emergency_direction	Emergency approach	category	none, NS, EW

For every field, the team must answer

- ☐ What does this field mean in one simple sentence?
- ☐ Is it a number, category, text identifier, input, target, or output?
- ☐ What is its unit?
- ☐ What values are allowed for the 20-row sample?
- ☐ What should happen when the value is missing?
- ☐ Which source or assumption justifies the final range?

Write the outputs separately

Required outputs: next_phase (NS/EW) and green_duration_sec, with safe minimum and maximum values

Source and 10,000-record plan

Document arrival/departure assumptions and safe green-time limits. Later use a reproducible simulator to produce at least 10,000 states.

Step 1 uses only a small sample. The final 10,000+ records/scenarios are created in Step 2, after the small sample and validation are correct.

Create the first sample carefully

Do not begin with 10,000 rows. First make a small file that every member understands.

How to create the sample

1	Create the file or files listed on the repository page.
2	Type the field names exactly as written in the data dictionary.
3	Create five rows for the mandatory cases shown below.
4	Add five ordinary rows with different valid values.
5	Add five boundary rows close to minimum or maximum values.
6	Add five difficult rows that combine conditions.
7	Keep one wrong row in a separate test file. Do not mix it into valid data.
8	Save as CSV using comma-separated values and a header row.

Five mandatory starter cases

Name	Example input	What to check
TRF-001	5, 5, 20, 20, 25, 22, none	Balanced low traffic
TRF-002	30, 6, 85, 25, 120, 30, none	Heavy NS queue
TRF-003	4, 28, 15, 80, 20, 115, none	Heavy EW queue
TRF-004	8, 12, 30, 40, 190, 65, none	Long waiting time despite shorter NS queue
TRF-005	20, 20, 60, 60, 90, 90, EW	Emergency must get priority

After entering the rows

☐	Open the CSV once in a text editor and once in a spreadsheet.
☐	Check that no heading contains an accidental space.
☐	Check that units are not mixed inside a numeric column.
☐	Check that categories use one spelling only.
☐	Check that identifiers are unique.
☐	Ask a member who did not create the file to explain three rows.

Write the comparison method

The baseline is deliberately simple. It lets you prove later whether the Soft Computing method is better.

1	Start with NS green for 30 seconds.
2	Then use EW green for 30 seconds.
3	Continue alternating, even when one queue is longer.
4	If emergency_direction is not none, allow an explicit safety override in the test version.
5	Reject negative queues, density over 100, or wait above the selected limit.

What to save

- [] Put the numbered rules in docs/STEP-1.md.
- [] Write the same rules as pseudocode in docs/baseline-pseudocode.md.
- [] For each of the five mandatory cases, write the expected baseline behavior.
- [] Mark every temporary threshold clearly.
- [] Do not call this baseline fuzzy logic, ANN, or GA.

Pseudocode format

```

READ one valid input row
IF the row is invalid:
  SHOW a clear error
ELSE:
  APPLY the simple rules in order
  CALCULATE the baseline output
  PRINT the output and the rule that was used
SAVE the result for later comparison

```

Step 1 requires the baseline design and expected behavior. A complete baseline program is built in Step 3, but writing a tiny runnable version now is encouraged.

Make the computer check your data

Install pandas and run the starter validator. The first version checks file shape, headings, missing values, and simple data types.

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
pip install pandas
pip freeze > requirements.txt
python src/validate_data.py
```

Starter content for src/validate_data.py

```
import pandas as pd

FILE = 'data/sample_input.csv'
REQUIRED = ['state_id', 'queue_ns', 'queue_ew', 'density_ns_pct', 'density_ew_pct', 'max_wait_ns_sec', 'max_wait_ew_sec', 'emergency_direction']
NUMERIC = ['queue_ns', 'queue_ew', 'density_ns_pct', 'density_ew_pct', 'max_wait_ns_sec', 'max_wait_ew_sec']

df = pd.read_csv(FILE)
print('shape:', df.shape)
print('columns:', list(df.columns))
print('missing values:', df.isna().sum().to_dict())

missing_columns = [c for c in REQUIRED if c not in df.columns]
assert not missing_columns, f'Missing columns: {missing_columns}'
assert not df[REQUIRED].isna().any().any(), 'Missing value found'
for column in NUMERIC:
    assert pd.api.types.is_numeric_dtype(df[column]), f'{column} must be numeric'

print('STEP 1 DATA CHECK PASSED')
```

Add project-specific checks

- ☐ Every percentage must stay inside 0 to 100.
- ☐ Counts, time, distance, power, money, or rainfall cannot be negative.
- ☐ Every category must match the documented spelling.
- ☐ Every identifier must be unique.
- ☐ The known invalid/impossible case must fail with a clear message.
- ☐ The five valid mandatory cases must pass.

Take a screenshot of one PASS result and one expected FAIL result. Save them in results/step1/.

Draw the screen before coding it

Use paper, PowerPoint, Figma, Canva, or a simple drawing tool. The sketch is not the final UI.

Your screen must show

- ☐ Input area: NS/EW queue, density, wait, emergency.
- ☐ One clear Run, Recommend, Predict, or Optimize button.
- ☐ Output area: Next phase, green seconds, baseline phase, queue chart, warning.
- ☐ A place for validation errors.
- ☐ A place for one explanation or chart.
- ☐ A visible difference between baseline output and future Soft Computing output.

Simple wireframe to copy

```
+-----+
| PROJECT TITLE                               |
+-----+
| INPUTS      | RESULTS                      |
| [field] [value] | Baseline: ...           |
| [field] [value] | Soft Computing: ...        |
| [field] [value] | Explanation: ...           |
| [ RUN / RECOMMEND ] | [ chart or visual area ] |
+-----+
| Validation / error message                   |
+-----+
```

Save the sketch

1	Export or photograph the sketch clearly.
2	Save it as docs/product-v1-sketch.png.
3	Insert it in docs/STEP-1.md.
4	Ask one person outside the team to explain what they think the screen does.
5	Correct any label that the person does not understand.

Make small, honest commits

Each commit should contain one understandable improvement. Every member must make at least one commit.

Recommended commit sequence

1	Create repository structure for SC04
2	Add team roles and Step 1 project contract
3	Define input fields, units, and validation rules
4	Document fixed-time traffic baseline
5	Add five mandatory test cases
6	Add sample-data validation script
7	Add Product V1 screen sketch
8	Add Step 1 evidence and run instructions

Before every push

- ☐ Pull the latest team version.
- ☐ Run the validation program.
- ☐ Check git status and add only intended files.
- ☐ Write a message that says what changed.
- ☐ Push your branch and ask another member to review.

Do not do this

- ☐ Do not use messages such as update, changes, final, or done.
- ☐ Do not upload the complete project in one last commit.
- ☐ Do not commit .venv, passwords, private datasets, or personal information.
- ☐ Do not overwrite another member's work without discussing it.

Exactly what to show faculty

Open the repository and demonstrate the following in order.

1	Show the repository URL and collaborator list.
2	Open README.md and state the one-sentence problem.
3	Open docs/STEP-1.md and explain the user, inputs, outputs, M1, and M2.
4	Open data/README.md and show the field meanings, units, source links, and assumptions.
5	Open the sample data and point out one normal, one boundary, and one difficult row.
6	Run <code>python src/validate_data.py</code> and show PASS.
7	Run or show the deliberately invalid case and explain the expected error.
8	Explain the baseline rules and expected result for one test case.
9	Show the Product V1 screen sketch.
10	Show Git history and one meaningful commit from every member.

Final Step 1 checklist

- ☐ Every required file exists and opens.
- ☐ The problem boundary is small and clear.
- ☐ Inputs and outputs have names, meanings, units, and rules.
- ☐ The starter data is understandable and valid.
- ☐ Five mandatory cases are written with expected behavior.
- ☐ The simple baseline is explainable.
- ☐ Validation shows both PASS and expected FAIL evidence.
- ☐ The screen sketch includes input, result, explanation, and errors.
- ☐ Every member has committed and can explain the work.

After faculty approval, begin Step 2: create the reproducible 10,000+ record/scenario pipeline. Do not change field meanings or units silently; update the contract and commit the change.

What comes after Step 1

Step	Stage	What happens
1	M1	Define one simple two-phase junction.
2	M1	Generate and validate traffic states.
3	M1	Build the fixed-time signal and queue updates.
4	M1	Build fuzzy phase priority and green duration.
5	M1	Create the working junction Product V1.
6	M2	Use GA to tune controller parameters.
7	M2	Compare controllers in difficult traffic cases.
8	M2	Deploy Product V2 and finish all evidence.

M1 finish line

- ☐ 10,000+ cited records or documented scenarios
- ☐ Working baseline
- ☐ Working core Soft Computing method
- ☐ Runnable Product V1
- ☐ At least two useful result visuals
- ☐ Interim report and M1 demonstration

M2 finish line

- ☐ Advanced or hybrid method
- ☐ Parameter experiments and difficult-case testing
- ☐ Baseline vs M1 vs M2 comparison
- ☐ Public deployment and usable interface
- ☐ Final report, approximately five-minute video, presentation, and viva preparation

A strong capstone is one that another person can open, run, understand, test, and compare.